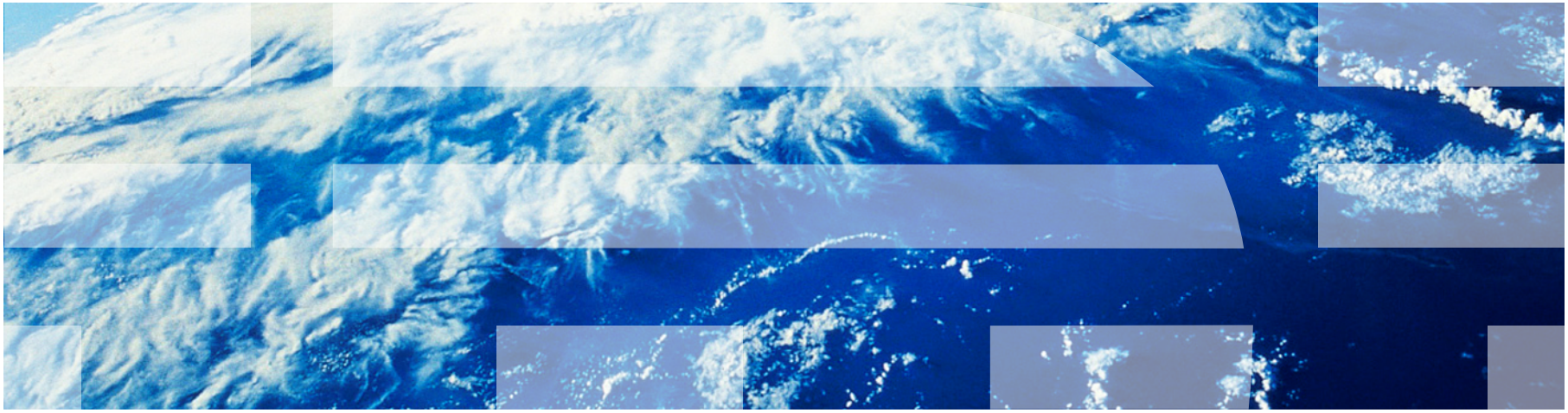
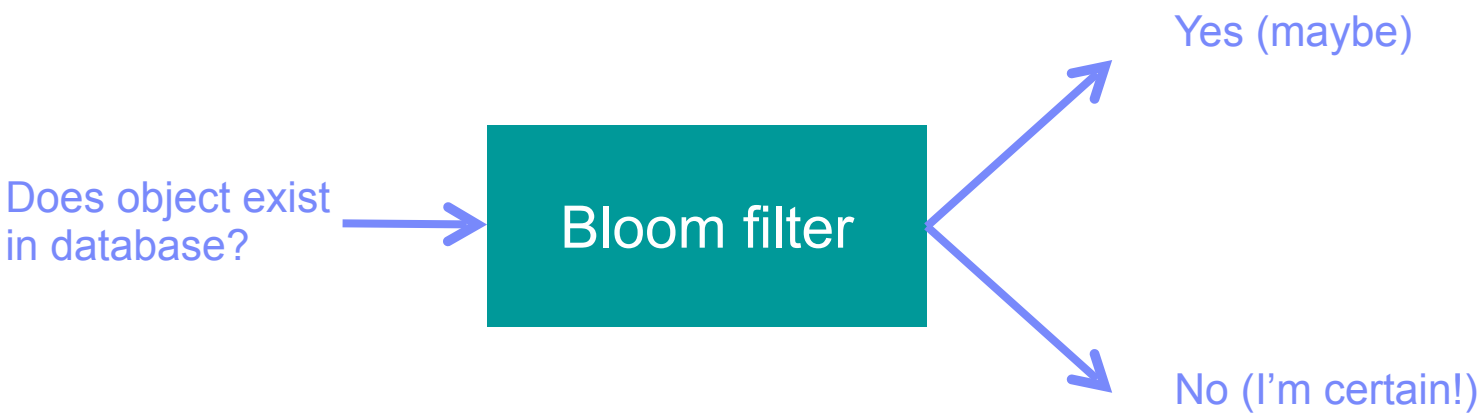
Bloom Filter



Motivation: Index doesn't fit in memory

- Example of a typical database:
  - 64GB memory
  - 2TB flash disk
  - Average key+value (e.g., entry) size: 50B → database can store up to 40 billion keys!
- A pointer needs to locate a key among 40 billion keys
  - Minimum pointer size: 36 bits ~ 4.5 bytes (let's round it to 5 bytes)
  - Index size = number of keys * byte rounded pointer size ~ 200GB
  - Does not fit in memory
- Therefore, we must use a multi-level index
  - Outer index in memory
  - Inner index on disk
- Minimum time reading a single object: 1 memory access + 2 flash accesses ~ 200us
- **What if object doesn't exist in the database?**
  - Still 200us!
- Can we do better?

Bloom filters [Bloom 1970]:
Approximate way to determine if object exists



Bloom filter parameters

- An array of m bits (can only be ‘0’ or ‘1’)
  - Array initialized to 0
- k independent **hash functions** $h_1,\ h_2, \ldots h_k$ that return a number between $1, \ldots\ m$

What is a hash function?

- What is a hash function?
  - $h(x) = y$, where x is an input (e.g., a key) and y is a uniformly random number
  - In our case, y is a random number between $1, .. \ m$

- If $h(x_1) = h(x_2)$, there is some probability that $x_1 = x_2$
- If $h(x_1) ! = h(x_2)$, we know **for sure** that $x_1 ! = x_2$

Algorithms: check membership and add membership

- To check if x is a member of the bloom filter, check whether $h_1(x), \ldots h_k(x)$ are all set to 1
  - If not, x is definitely not a member
  - If yes, x might be a member
    - There can be false positives!

- To add x, set the positions of $h_1(x), \ldots h_k(x)$ all to 1
  - Some positions might already have been set as 1

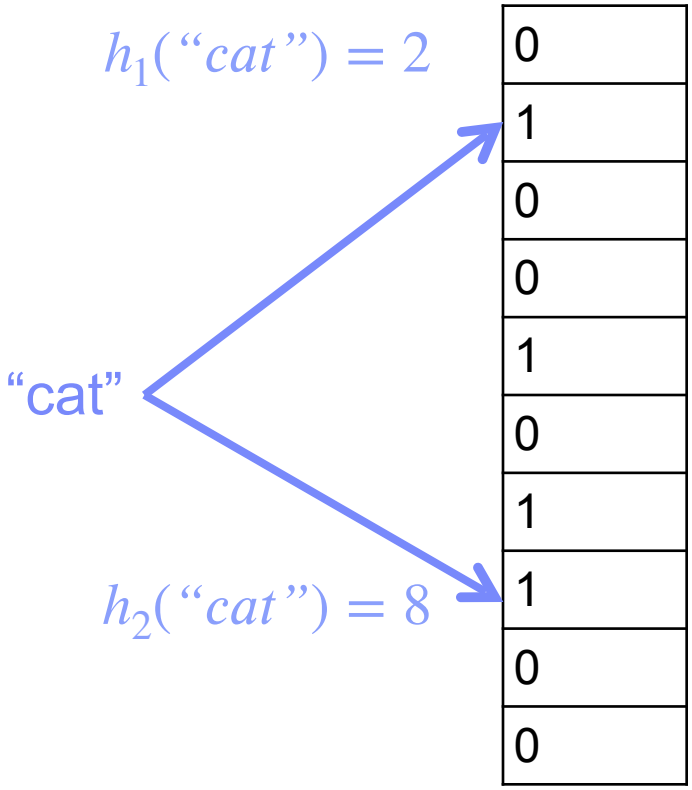
How does a bloom filter work? Example

m = 10
k = 2

0
1
0
0
1
0
1
1
0
0

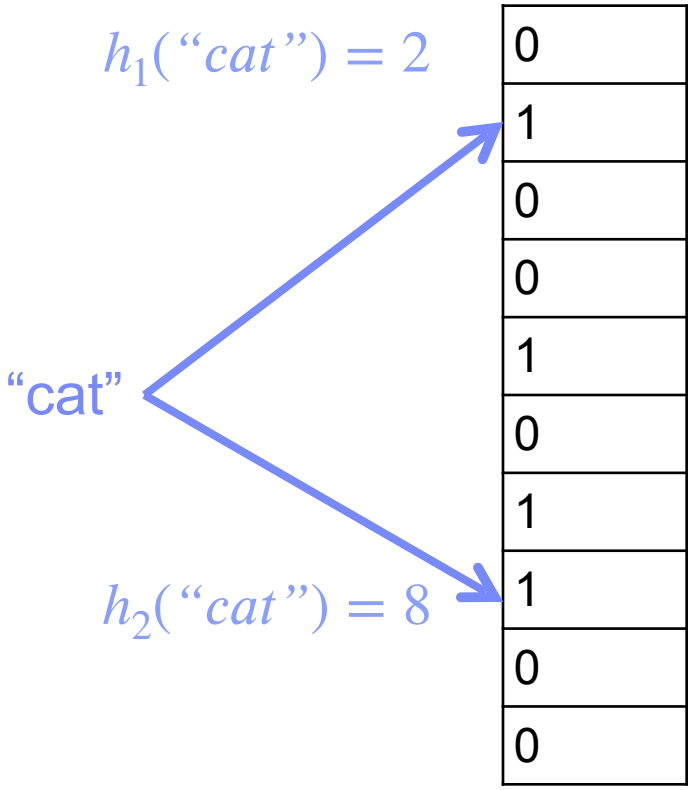
Is cat in DB?

m = 10
k = 2



Is cat in DB? Maybe!

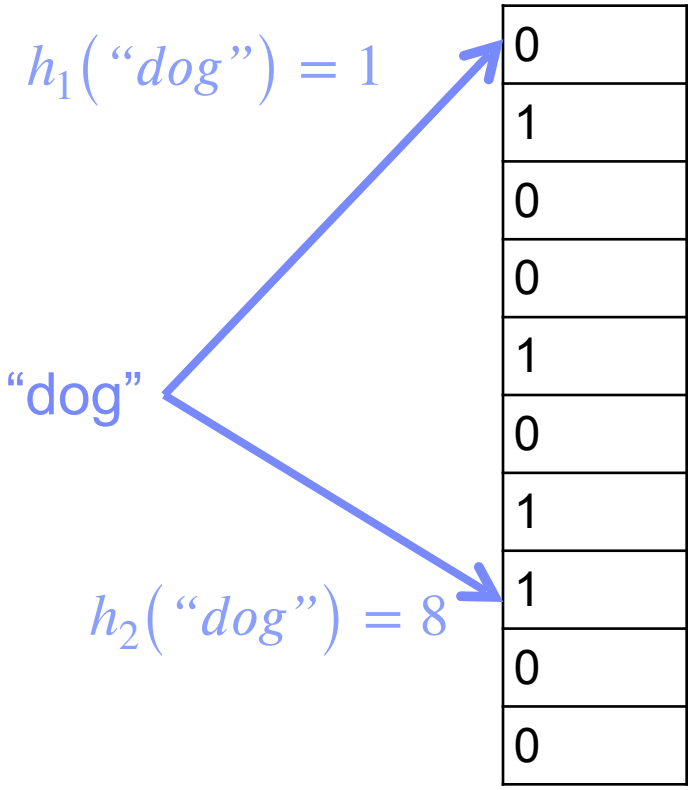
m = 10
k = 2



Cat might exist in DB!

Is dog in DB?

m = 10
k = 2



Dog definitely does not exist in DB

→ We don’t need to read from disk

Add dog to DB

m = 10
k = 2

$h_1(\text{“}dog\text{”}) = 1$

“dog”

$h_2(\text{“}dog\text{”}) = 8$

1
1
0
0
1
0
1
1
0
0

Goal: low false positives

- Define n as the total number of unique objects that might ever be inserted into the database
  - For example, for a bank database that uses account ID as keys, this is the total number of accounts the bank will ever have
- Let's assume $kn < m$